Hierarchical Neural Skinning Deformation with Self-supervised Training for Character Animation

TIANYI WANG, Tianjin University, China
SHIGUANG LIU*, Tianjin University, China

We propose a self-supervised neural deformation method for character skin deformation animations, incorporating musculoskeletal and adipose tissue deformations driven by joint movements. This hierarchical approach combines two self-supervised neural emulators: the musculoskeletal neural emulator, based on a biomaterial musculoskeletal model, and the soft-tissue neural emulator, capturing soft secondary dynamics. The neural emulators achieve self-supervised learning through a loss function derived from physical principles, eliminating the need for ground-truth datasets. The musculoskeletal neural emulator enhances deformation effects, including musculoskeletal bulging, contraction, and dynamic responses, and generates inner-layer animations. Building on this, the soft-tissue neural emulator adds enhanced deformation effects, such as secondary motion, driven by the inner-layer animations.During inference, the method efficiently simulates realistic skin deformation animations for arbitrary character structures and joint motion sequences. The resulting animations demonstrate visual fidelity comparable to, or surpassing, state-of-the-art data-driven methods in the industry.

CCS Concepts: • **Computing methodologies** → **Physical simulation**; *Neural networks*.

Additional Key Words and Phrases: self-supervised learning, skinning animation, neural emulator

ACM Reference Format:
Tianyi Wang and Shiguang Liu. 2025. Hierarchical Neural Skinning Deformation with Self-supervised Training for Character Animation. *Proc. ACM Comput. Graph. Interact. Tech.* 8, 1, Article 8 (May 2025), 20 pages. https://doi.org/10.1145/3728300

1 Introduction

Efficient 3D character animation methods have long been central to computer graphics and digital entertainment. Skinning techniques, mapping skeletal transformations to mesh vertices, enable real-time interactions—particularly Linear Blend Skinning (LBS) [Magnenat et al. 1988], renowned for its stability and parallel GPU capability. However, geometry-based methods cannot capture biomechanical details like muscle deformations or fat wobbling.

Physics-based simulations improve realism by incorporating biomechanical properties. Finite Element Methods (FEM) [Gourret et al. 1989] accurately simulate bones, muscles, and fat but are too slow for real-time use. Position-Based Dynamics (PBD) supports faster multilayer soft body simulations, though its simplified constraints limit complex muscle behaviors.

Neural networks bridge fidelity and performance by approximating physics. Recent deep learning methods map skeleton transformations to vertex deformations [Anguelov et al. 2005; Bailey et al.

*Corresponding author.

Authors' Contact Information: Tianyi Wang, Tianjin University, Tianjin, China, tju_wty@qq.com; Shiguang Liu, Tianjin University, Tianjin, China, lsg@tju.edu.cn.

ACM 2577-6193/2025/5-ART8
https://doi.org/10.1145/3728300

2018; Han et al. 2024; Li et al. 2021], producing visually rich soft tissue animation but depending heavily on curated training datasets.

We present an unsupervised hierarchical framework for soft body animation via a graph-based network. Our method features neural emulators for skin/fat and muscle layers—both powered by a Multi-Scale Edge Aggregation Mesh Graph Network (MSEA-MGN). By formulating constrained secondary motions as an energy optimization problem, and introducing biomechanical material models as incremental potential energy supervision, we eliminate the need for ground-truth data.

Leveraging graph networks, our solution generalizes across arbitrary mesh topologies, enhancing scalability and robustness while delivering high-quality soft body animations without reliance on expensive datasets.

2 Related Work

This section introduces the works related to this study. Section 2.1 reviews traditional geometry-based methods and physics-based methods for character deformation. Section 2.2 discusses advanced deep learning-based methods for character soft body deformation, which have achieved remarkable results.

2.1 Geometry-Based and Physics-Based Methods

Linear Blend Skinning (LBS) [Magnenat et al. 1988] and Dual Quaternion Skinning (DQS) [Lee et al. 2013] remain foundational techniques for real-time mesh deformations in games, using per-vertex skinning weights. Pose Space Deformation (PSD) [Lewis et al. 2023; Merry et al. 2006; Wang and Phillips 2002] extends LBS with example-based pose corrections. Mancewicz et al. [Mancewicz et al. 2014]proposed a Laplacian smoothing algorithm (delta-mush) to mitigate artifacts in joint deformation. Geometric approaches also model muscle deformation: Richer et al. [Yilmaz 2009] relied on ellipsoidal implicit surfaces, Wilhelms et al. [Wilhelms and Van Gelder 1997] used generalized cylinders with cross-sectional scaling, and Scheepers et al. [Scheepers et al. 1997] employed tubular patches with scaling and tension for isometric contraction. Other work provides anatomically accurate musculoskeletal models [Aubel and Thalmann 2001; Fung 2013].

Geometric methods alone cannot fully capture detailed muscle deformations. Physics-based approaches address this by modeling skeletal-driven muscle behavior and its effects on fat and skin. Chen and Zeltzer [Chen and Zeltzer 1992] pioneered FEM-based muscle simulations for graphics, while Dao and Tho [Dao and Tho 2018] surveyed hyperelastic models in biomechanics. Teran and Sifakis [Teran et al. 2005] introduced a simplified FEM for soft tissue beneath elastic skin. Modi et al. [Modi et al. 2020] proposed a deformation gradient/displacement formulation for efficient quasi-static musculoskeletal simulations. Smith et al. [Smith et al. 2018] employed an isotropic Neo-Hookean model for skeleton-driven flesh.

2.2 Deep Learning-Based Methods

Neural networks can effectively compress high-precision data and predict new cases, mitigating stability and computational cost issues. End-to-end approaches predict dynamic behaviors directly from mesh inputs without user involvement in intermediate physical simulations. For instance, Bailey et al. [Bailey et al. 2018] incorporated nonlinear deformations into real-time skinning, Holden et al. [Holden et al. 2019] employed an autoencoder to approximate simulations in a reduced space, and Li et al. [Li et al. 2021] introduced Neural Blend Shapes for pose-dependent deformations.

Data-driven methods often bypass explicit musculoskeletal modeling by learning skin deformations from observations. Anguelov et al. [Anguelov et al. 2005] proposed SCAPE to handle body shape and pose variation, inspiring subsequent motion capture work [Chen et al. 2013; Pons-Moll et al. 2015].

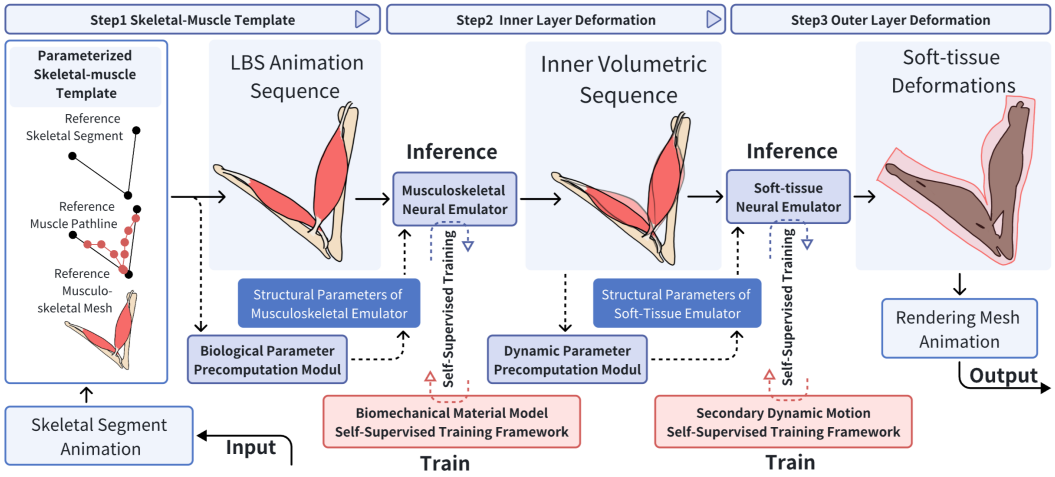

Fig. 1. Overview of the proposed hierarchical approach

For more detailed effects, recent approaches model individual material layers such as fat and skin [Bailey et al. 2018; Jin et al. 2022a; Santesteban et al. 2020], or train mass-spring parameters for skeletal-based skin deformations [Park and Hodgins 2008]. Others incorporate muscle and tendon simulations [Chentanez et al. 2020a; Li et al. 2022]. Recent studies have explored employing neural networks to provide trained delta correctives for Linear Blend Skinning (LBS). These approaches integrate neural networks into rigging systems[Chentanez et al. 2020b; Jin et al. 2022b]. Loper et al. [Loper et al. 2015] proposed the Skinned Multi-Person Linear (SMPL) model, a parameterized human body framework that enhances LBS realism through machine learning techniques. Building upon SMPL, Santesteban et al. further incorporated soft tissue deformation dynamics [Santesteban et al. 2020]. EPIC [Han et al. 2024] employs an Active Neural Network for different muscle activation states, thereby reducing ground-truth dependencies.

Graph Neural Networks (GNNs) have also proven effective for dynamic physics simulations[Sanchez-Gonzalez et al. 2020; Santesteban et al. 2022]. Pfaff et al. [Pfaff et al. 2020] presented Mesh Graph Nets, which propagate dynamics through recurrent message-passing for fluids, cloth, and soft bodies. Building on self-supervised strategies, we extend such methods to character skinning deformation for the first time, eliminating the need for ground-truth datasets.

3 Method

The motion of skeletal joints is accompanied, on one hand, by the activation of muscle fibers, resulting in the bulging and contraction of skeletal muscles; on the other hand, by the dynamic properties of isotropic materials such as tendons, leading to movement. Additionally, the outer fat layer undergoes secondary dynamic deformation driven by the motion of the inner skeletal muscles.

To address these phenomena, we constructed two neural emulators (the musculoskeletal neural emulator and the soft-tissue neural emulator, as shown in Fig. 1). These emulators are applied in the three steps illustrated in Fig. 1 to map joint motion sequences to rendering mesh deformations.

Step 1: Skeletal-Muscle Template We introduce a parameterized skeletal-muscle template (Appendix A.1) that adapts to any bipedal 3D character, generating an LBS animation sequence for both skeletal and muscle meshes. By vertex blending, the volumetric mesh sequence is

also derived. A biological parameter precomputation module (Appendix A.2) then extracts muscle-specific information from the volumetric meshes, assembling it into a structured parameter vector (Section 3.1) for use in the next step.

Step 2: Inner Layer Deformation A trained musculoskeletal neural emulator processes the volumetric mesh sequences and the parameter vector, inferring deformations driven by muscle activation and tendon dynamics. The resulting inner layer animation is stored as the character's volumetric mesh sequence. A dynamics precomputation module additionally generates secondary dynamics parameters, stored in a new structured parameter vector (Section 3.1).

Step 3: Outer Layer Deformation A trained soft-tissue neural emulator uses the inner layer mesh sequence and the parameter vector to predict soft-tissue deformations dominated by secondary dynamics. The final outer mesh deformations are then blended at the vertex level to yield the rendering mesh animation.

Section 3.1 introduces the model architectures of the two neural emulators. Furthermore, we presents a supervisory framework based on physical information and biomechanical models to enable the self-supervised training of the two neural emulators, detailed in Sections 3.2 and 3.3, respectively.

3.1 Model Composition

The graph network models the 3D mesh as nodes connected by edges and organizes it into a hierarchically nested multi-scale structure. At each level, central vertices are selected, and vertices with even adjacency are extracted to form coarser levels connected by bi-stride structured edges [Cao et al. 2023]. This iterative process creates a multi-layer nested graph with shared nodes across scales.

Encoders process structured parameter vectors encoding physical, geometric, and biomaterial properties, stored on the graph's nodes and edges. Tables 1 list the required parameters for each emulator.

The musculoskeletal and soft-tissue neural emulators introduced in this work are constructed following the physics-informed graph neural network (PIGNN) paradigm—a widely adopted encoder-processor-decoder architecture in computational biomechanics. Within this framework, input features undergo sequential processing through these components, with the decoder ultimately generating physically meaningful predictions. The architectural workflow proceeds as follows.

As shown in Fig. 2, both emulators share a similar architecture for their encoders and decoders.

The encoder processes parameter vectors from the graph's nodes and edges into embedding features. It includes a node encoder and multiple edge encoders, corresponding to different graph scales. These encoders are implemented as Multi-Layer Perceptrons (MLPs) consisting of fully connected layers, ReLU activations, and layer normalization.

In the musculoskeletal neural emulator, nodes use 17-dimensional parameter vectors, and each layer of coarsening edges uses 10-dimensional parameter vectors (see Table 1, musculoskeletal section). For the soft-tissue neural emulator, nodes use 12-dimensional parameter vectors, and each coarsening edge layer uses 11-dimensional parameter vectors (see Table 1, soft-tissue section).

In the node encoder, parameter vectors are divided into physical and geometric components, each processed by an independent MLP with a 64-dimensional output. The resulting embeddings are concatenated to form a 128-dimensional node embedding. In the edge encoder, each coarsening layer uses an independent MLP with a 128-dimensional output to compress the parameter vectors of the corresponding coarsening edges, generating 128-dimensional edge features.

Table 1. Structural parameters of the emulator

Category	Physical Quantity	Description
	Musculoskeletal Emulator	
Node	Velocity v	Velocity of the node at a previous time step
	Normal vector $\mathbf{n}$	Normal vector of the node
	Node mass m	Mass of the node
	Muscle activation state α	Muscle activation degree
	Node material type a_0	One-hot encoding of the node's material type (e.g., muscle, tendon)
	Final layer L	One-hot encoding of the node's final layer
Edge	Relative position $\Delta\mathbf{x}_{ij}$	Relative position of the edge at the current time step $\Delta\mathbf{x}_{ij} = (\mathbf{x}_i - \mathbf{x}_j)$
	Relative position magnitude $\|\Delta\mathbf{x}_{ij}\|$	Magnitude of the relative position of the edge at the current time step
	Relative position in rest state $\Delta\mathbf{x}_{ij}^{\text{rest}}$	Relative position of the edge in the rest state
	Relative position magnitude in rest state $\|\Delta\mathbf{x}_{ij}^{\text{rest}}\|$	Magnitude of the relative position of the edge in the rest state
	Muscle activation state α_{ij}	Average activation state of the muscles connected to the edge
	Node material type parameter M	Average material type parameter of the nodes on the edge
	Soft-Tissue Emulator	
Node	Velocity v	Velocity of the node at a previous time step
	Normal vector n	Normal vector of the node
	Node mass m	Mass of the node
	Material parameter μ_{Lame}	Lame constant μ
	Material parameter α_{Lame}	Lame constant α
	Material parameter $\alpha_{Damping}$	Damping parameter α
	Material parameter $\beta_{Damping}$	Damping parameter β
	Node type w	Unique encoding of the node type: primary, secondary partition regions
	Final depth L	Unique encoding of the node's final depth layer
Edge	Relative displacement Δu_g	Relative displacement of the edge at the current time step
	Relative displacement in rest state Δu_{agg}	Relative displacement of the edge in the rest state
	Lame constant μ_{agg}	Average Lame constant μ of the edge
	Lame constant α_{agg}	Average Lame constant α of the edge
	Material parameter α_{agg}	Average damping parameter α of the edge
	Material parameter β_{agg}	Average damping parameter β of the edge

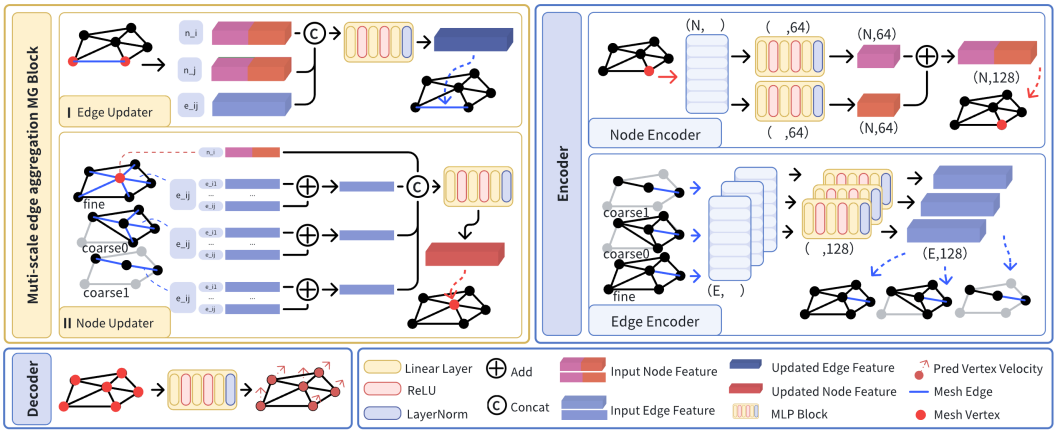

Fig. 2. The architecture of the encoder, processor, and decoder

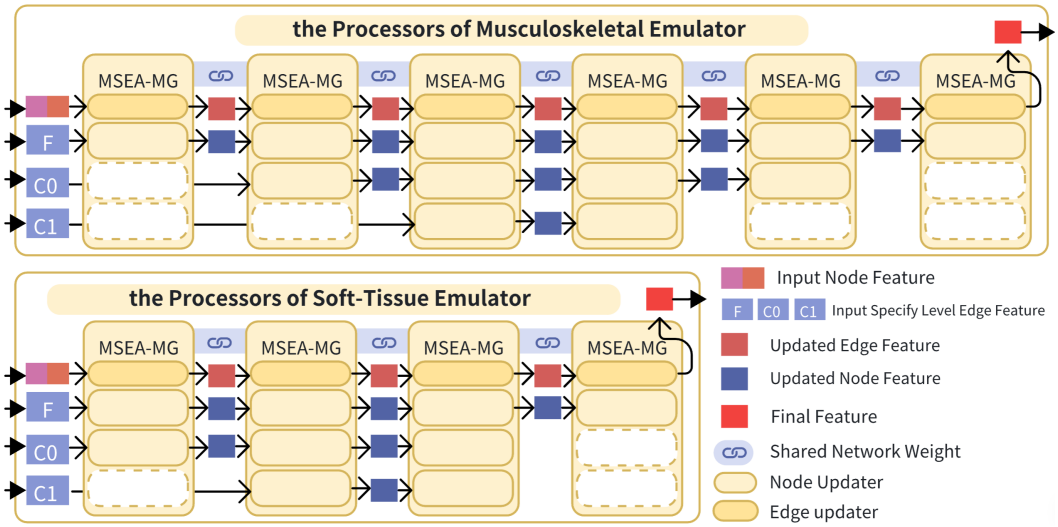

Fig. 3. The construction of the processors for the two emulators

The decoder, composed of MLPs similar to those in the encoder, takes 128-dimensional node embeddings as input and outputs 3-dimensional velocity increments for the nodes. These outputs serve as the graph network's predictions, providing clear physical significance.

The processor receives the encoded embeddings and utilizes an information propagation mechanism to model interactions and updates between nodes and edges. As depicted in the upper-left quadrant of Fig. 2, the processor architecture operates through the following mechanism: Edge-embedded features across multiple coarsening scales (denoted as F, C0, C1 in Fig. 2 left panel) are initially processed by node updaters to refresh the central node's feature representation (highlighted by the red node in the node updater schematic). Subsequently, edge updaters recompute edge features by aggregating updated nodal states from adjacent vertices. A single iteration of alternating node-edge updates constitutes a message passing step [Dalton et al. 2023], where all

mesh vertices undergo synchronized feature propagation under finite-radius constraints—each step only incorporates neighboring nodes.

To overcome this perceptual limitation, our previously proposed multi-scale edge aggregation (MSEA) modules [Wang and Liu 2024] enable hierarchical information routing across coarsening scales, effectively expanding the single-step receptive field. Achieving physics-compliant signal propagation across extended neighborhoods necessitates cascading multiple MSEA-MG modules, with the stacking configuration (Fig. 3) being task-optimized through energy-accuracy tradeoff analysis. Notably, edge features at underutilized coarsening levels (dashed-box edge updaters in Fig. 3) bypass current-step computation and propagate directly to subsequent iterations, preserving multi-resolution feature continuity while minimizing redundant processing. As previously discussed, each MSEA-MG module implements distinct feature update steps. The weight-sharing mechanism illustrated in Fig. 3 fundamentally denotes that all MSEA-MG modules employ identical neural network parameters, differing solely in their hierarchical selection of coarsening-level edge features (i.e., which multiscale edge embeddings are activated per module). Different stacking strategies are applied for the two emulators, as shown in Fig. 3.

Throughout the feature processing pipeline of the emulators, all computational operations occur at the vertex level of character meshes. This architectural paradigm enables the trained model to generate predictions for character meshes of arbitrary topological structures, provided that the encoder receives the required structured parameter vectors (as specified in Table 1) . This design confirms that both emulators exhibit cross-character generalization capabilities rooted in their topological agnosticism.

3.2 The Self-Supervised Training Framework for the Musculoskeletal Neural Emulator

To enable the self-supervised training of the musculoskeletal neural emulator, we introduce a physics-inspired supervision framework based on a biomechanical material model. Building on previous work[Wang and Liu 2024], the solution of the energy system can be modeled as a minimization objective optimization function from the perspective of incremental energy. For a dynamic system composed of muscle finite elements, the objective function for solving its quasi-static equilibrium state can be expressed as:

$$x^{t+1} = \arg\min_{x} \left(\frac{1}{2} \mathbf{a}^{\top} \mathbf{M} \mathbf{a} + \Phi_{\text{Muscle Strain}}(x) + \mathbf{G}(x) \right), \quad \text{subject to } \mathbf{S} x^{t+1} = \chi_{\text{LBS}}(t+1), \tag{1}$$

where x represents the vertex coordinates at the current time t, x^{t+1} denotes the predicted vertex coordinates at the next time step, $\mathbf{a}$ represents the vertex acceleration, $\mathbf{M}$ is the finite element mass matrix, $\Phi_{\text{Muscle Strain}}$ represents the muscle strain potential energy term, and $\mathbf{G}$ denotes the gravitational potential energy term.

By using methods such as gradient descent, the neural network can learn predictions that align with the behavior of biological materials during the optimization of the objective function. Centered around the objective function, we construct the corresponding physical energy loss function based on biomechanical materials $\mathcal{L}_{\text{Bio-physical Loss}}$:

$$\mathcal{L}_{\text{Bio-physical Loss}} = \mathcal{L}_{\text{Inertia}} + \mathcal{L}_{\text{Gravity}} + \mathcal{L}_{\text{Muscle Strain}} \tag{2}$$

The inertial energy term $\mathcal{L}_{\text{inertia}}$ and the gravitational potential energy loss term $\mathcal{L}_{\text{gravity}}$ are calculated in the same manner as in previous works[Wang and Liu 2024]. The muscle strain potential energy loss term $\mathcal{L}_{\text{Muscle Strain}}$ is the key factor in supervising the predictions of the musculoskeletal neural emulator to ensure compliance with the deformation patterns of biomechanical materials.

The muscle strain potential energy loss term is defined as:

$$\mathcal{L}_{\text{Muscle Strain}} = \sum_{el=1}^{n} \Psi_{\text{Muscle}} V_{el}, \tag{3}$$

The strain potential energy density function Ψ_{Muscle} represents the internal potential energy generated by the deformation of the muscle mesh from its reference configuration to its current configuration. According to prior work [Röhrle et al. 2017], Ψ_{Muscle} is defined as:

$$\Psi_{\text{Muscle}} = \Psi_{\text{Passive}} + \alpha \gamma^M \Psi_{\text{Active}} \tag{4}$$

where α represents the activation state parameter of the muscle fibers, $\alpha = 0$ indicates fully deactivated muscle, and $\alpha = 1$ indicates maximum muscle activation. γ^M denotes the material type parameter, $\gamma^M = 1$ represents an element entirely composed of muscle material, and $\gamma^M = 0$ represents an element entirely composed of tendon material. Both α and γ^M are provided by the biological parameter precomputation module described in Section 3.1.

Ψ_{passive} is the passive strain potential energy function, which describes the strain potential energy generated by deformation when the muscle is in a deactivated state. In this section, we adopt the J-shaped force-stretch material behavior introduced by Markert et al. [Markert et al. 2005], defining the passive strain potential energy function using a polynomial expression:

$$\Psi_{\text{Passive}} = \begin{cases} \frac{c_3}{c_4}\left(\lambda_f^{c_4} - 1\right) - c_3 \ln \lambda_f, & \text{if } \lambda_f \geq 1, \\ 0, & \text{else.} \end{cases} \tag{5}$$

Among these parameters, c_3 and c_4 are biomechanical material coefficients measured by Zheng et al. [Zheng et al. 1999] and Weiss et al. [Weiss and Gardiner 2001]. c_i^M represents the biomechanical material coefficient for muscles, and c_i^T represents the biomechanical material coefficient for tendons. The composite biomechanical material coefficient of the finite element is interpolated using the material type parameter γ^M as follows:

$$c_i = \gamma^M c_i^M + (1 - \gamma^M) c_i^T \tag{6}$$

λ_f denotes the fiber stretch, defined as $\lambda_f = \sqrt{I_4}$, where I_4 is the fourth invariant of the stress tensor, defined as $I_4 = \text{tr}(MC)$. C is the right Cauchy-Green deformation tensor, defined as $C = F^T F$, where F is the deformation gradient, which can be constructed from the coordinates of finite element vertices. M is the structural tensor, defined as $M = \mathbf{a}_0 \otimes \mathbf{a}_0$, where $\mathbf{a}_0$ represents the fiber direction parameter of the finite element, and $\otimes$ denotes the tensor product operation.

Ψ_{active} represents the active strain potential energy function, describing the strain potential energy generated during deformation when the muscle is in an activated state. In this section, we adopt the bell-shaped force-stretch behavior material model introduced by Hatze et al. [Hatze 1978], defined as:

$$\Psi_{\text{Active}} = \begin{cases} -\frac{S_{\max}}{v_{\text{asc}}} \left(\lambda_f^{\text{opt}} \Delta W_{\text{asc}}\right)^{-1/v_{\text{asc}}} \exp\left(-\left|\frac{\lambda_f/\lambda_f^{\text{opt}} - 1}{\Delta W_{\text{asc}}}\right|^{v_{\text{asc}}}\right), & \lambda_f \leq \lambda_f^{\text{opt}}, \\ -\frac{S_{\max}}{v_{\text{desc}}} \left(\lambda_f^{\text{opt}} \Delta W_{\text{desc}}\right)^{-1/v_{\text{desc}}} \exp\left(-\left|\frac{\lambda_f/\lambda_f^{\text{opt}} - 1}{\Delta W_{\text{desc}}}\right|^{v_{\text{desc}}}\right), & \lambda_f > \lambda_f^{\text{opt}}. \end{cases} \tag{7}$$

where λ_{opt}^f represents the optimal length constant of the muscle fiber, and $S_{\max}$ denotes the maximum stress constant generated by skeletal muscles under optimal length stretching. v_i and ΔW_i are constants representing the steepness and width of the Gaussian bell curve, respectively, with corresponding values for the ascending phase ($i := \text{asc}$) and descending phase ($i := \text{desc}$). The

Table 2. Biomaterial constants and their references

Biomaterial Constant	Value	Reference
c_3^M	4.02×10^{-7} MPa	Zheng et al. [Zheng et al. 1999]
c_4^M	38.5 (–)	
c_3^T	7.99 MPa	Weiss and Gardiner et al. [Weiss and Gardiner 2001]
c_4^T	16.6 (–)	
ΔW_{asc}	0.25 (–)	
ΔW_{desc}	0.15 (–)	
v_{asc}	4.0 (–)	Günther et al. [Siebert et al. 2014]
v_{desc}	3.0 (–)	
λ_f^{opt}	1.35 (–)	
S_{max}	0.30 MPa	

values and sources of the biomechanical material constants used in this section are detailed in Table 2.

3.3 The Self-Supervised Training Framework for the Soft-tissue Neural Emulator

Similar to the approach described in Section 3.2 and in previous work[Wang and Liu 2024] on secondary dynamics deformers, the self-supervised training framework for the soft-tissue neural emulator is constructed by reformulating the solution of the secondary dynamic motion equations as an optimization problem with respect to the predicted positions:

$$x^{t+1} = \arg\min_x \left(\frac{1}{2}\mathbf{a}^\top \mathbf{M}\mathbf{a} + \frac{1}{2}\mathbf{v}^\top \mathbf{D}\mathbf{v} + \Phi(x) + \mathbf{G}(x) \right), \quad \text{subject to } \mathbf{S}x^{t+1} = \chi_{\text{Inner}}(t+1). \tag{8}$$

x represents the coordinates of the point set of the 3D character at the current time step. M is the mass matrix. x^{t+1} denotes the secondary animation prediction made by the emulator. D represents the damping matrix. S is the selection matrix, where $Sx^{t+1} = \chi_{\text{Inner}}(t+1)$ indicates that the vertices selected by the selection matrix are updated to the inner layer coordinates at the next time step. a represents acceleration, v represents velocity, and $\mathbf{G}$ denotes gravitational potential energy.

The objective function is further constructed as the physics-inspired self-supervised loss function $\mathcal{L}_{\text{Physical-Loss}}$, which is used to enable the self-supervised training of the neural emulator:

$$\mathcal{L}_{\text{Physical-Loss}} = \mathcal{L}_{\text{Inertia}} + \mathcal{L}_{\text{Gravity}} + \mathcal{L}_{\text{Strain}} + \mathcal{L}_{\text{Damping}}. \tag{9}$$

The inertial potential energy loss term ($\mathcal{L}_{\text{inertia}}$), gravitational potential energy ($\mathcal{L}_{\text{gravity}}$), and strain potential energy loss term ($\mathcal{L}_{\text{strain}}$) are calculated using methods consistent with previous works[Wang and Liu 2024].

The Rayleigh damping potential energy loss term ($\mathcal{L}_{\text{damping}}$) reflects the energy dissipation caused by material motion. In this section, a linear elastic model is used for its construction [Rao 2010]:

$$\mathcal{L}_{\text{Damping}} = \frac{1}{2}\mathbf{v}^\top (\alpha\mathbf{M} + \beta\mathbf{K})\,\mathbf{v}. \tag{10}$$

where $\mathbf{M}$ is the mass matrix, computed as a lumped mass matrix with diagonal elements representing the mass m_i of the corresponding nodes, defined as $m_i = \rho \mathbf{v}_e/4$. Here, ρ is the material density,

and $\mathbf{v}_e$ is the volume of the finite element. α and β are the material damping coefficients. $\mathbf{K}$ is the stiffness matrix, with its assembly method detailed in Appendix A.3.

4 Experiments

Using the parameterized skeletal-muscle model defined in Appendix A.1, we constructed three human structural models—torso, upper limbs, and lower limbs—to generate mesh data for the training dataset. The reference data, including skeletal and muscle meshes, are derived from the Visible Human Male dataset [Spitzer et al. 1996] and associated models (e.g., skeletal segments and muscle path points). By randomly sampling skeletal segment lengths L within a specified range, we generated five models each for the upper limbs, torso, and lower limbs. Skeletal animation data were extracted from the KIT dataset in AMASS [Mahmood et al. 2019] and mapped to corresponding body parts via animation retargeting, resulting in a total of 80,000 frames. The surface mesh animation sequences from the parameterized skeletal-muscle models, driven by skeletal animation, were converted into volumetric mesh sequences using a vertex-blend skinning script developed with the Blender Python API [Blender Foundation 2024].

The experiments were conducted on a desktop workstation equipped with an Intel i7-10700 CPU and an Nvidia RTX 3060 GPU. The model framework and training were implemented using the deep learning library PyTorch [Facebook AI Research 2024], with Blender 3.0 LTS [Blender Foundation 2024] employed for visualization and geometric processing. Training began with single-step predictions, incrementally adding one step every 20,000 iterations, up to three-step predictions. The Adam optimizer was utilized with an initial learning rate of 5×10^{-5}, decaying by a factor of 0.999 every 5,000 iterations. Training batches consisted of one sample per batch and stabilized after 628,000 iterations over 35 hours.

Roll-RMSE was used as an objective metric to evaluate visual fidelity [Zheng et al. 2021], serving as the primary comparison metric. The neural soft-tissue emulator training adhered to methods outlined in previous works[Wang and Liu 2024].

4.1 Comparison experiments with the state-of-the-art methods

Samples from an unseen range of skeletal lengths, not included in the training data, are used to construct validation upper limb models for the experiments. Skeletal segment animations are extracted from the upper limb portion of the DanceDB dataset [Mahmood et al. 2019], representing validation motion data unseen during training.

The Ziva muscle simulation toolkit [Ziva Dynamics 2024] is used to simulate muscle dynamics with the same input data (skeletal segments, skeletal mesh, muscle mesh) as the validation upper limb models. Muscle simulation results from the validation motion set serve as ground truth for the experiments.

The proposed neural musculoskeletal emulator is compared against Neural Blend Shape (NeuralBS) [Li et al. 2021], CFD-GCN [Belbute-Peres et al. 2020], GNS [Mahmood et al. 2019], and Linear Blend Skinning (LBS) [Magnenat et al. 1988]. All methods use identical validation upper limb models and three 50-frame validation motions (Motion 01, Motion 02, and Motion 03) sampled from the validation motion set. The rendered results are evaluated under consistent rendering settings using the Roll-RMSE metric [Zheng et al. 2021] to quantify visual fidelity across methods.

In the second column of Figure 4, frame captures of the rendering sequence from the proposed method highlight a significant contraction effect in the biceps brachii during arm flexion. This reflects the neural musculoskeletal emulator's ability to respond accurately to activation signals, mapping them to finite element muscle expansions perpendicular to the muscle fiber direction, and aligning with the behavior of biomechanical materials. In contrast, the NeuralBS method shows poor performance for this validation motion, with predicted neural shape keys resembling linear

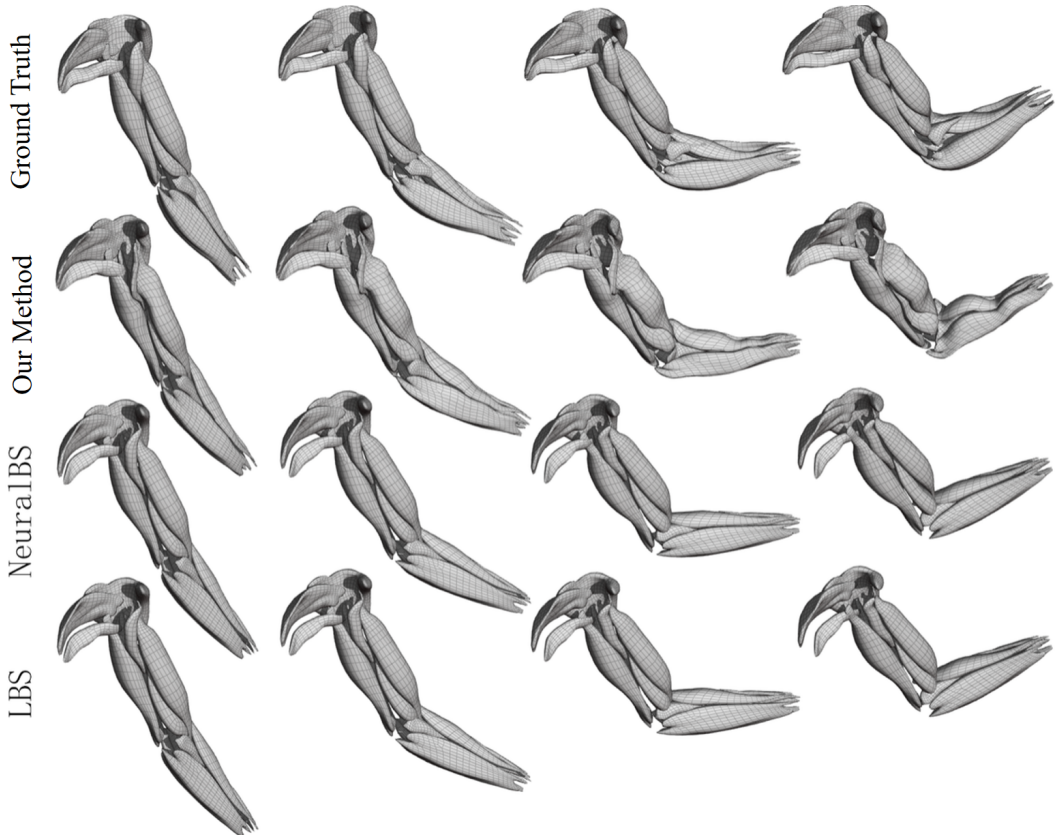

Fig. 4. Comparative experiments on the neural musculoskeletal emulator

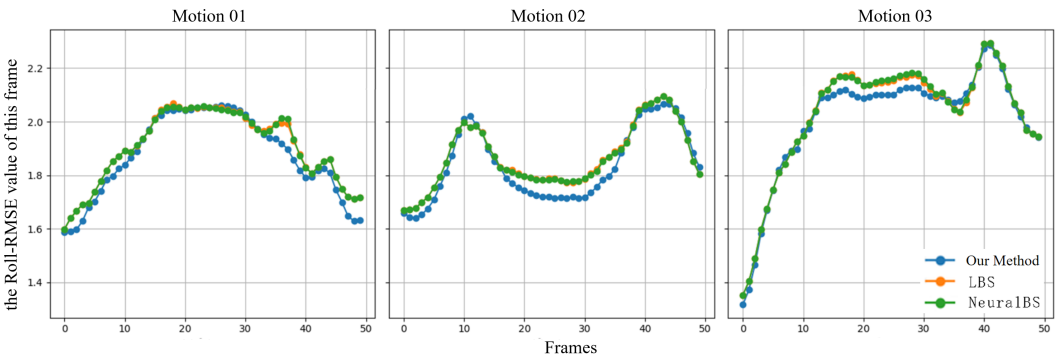

Fig. 5. Roll-RMSE metric curve for the comparative experiments of the neural musculoskeletal emulator

blend skinning and failing to exhibit noticeable responses such as biceps brachii expansion during arm flexion.

Figure 5 compares the roll-RMSE performance of the proposed Musculoskeletal Emulator, NeuralBS, and Linear Blend Skinning (LBS) across frames in validation motions 01, 02, and 03. The line

Table 3. Roll-RMSE performance of neural musculoskeletal emulator in comparative experiments

Sample|RMSE↓	Our Method	NeuralBS	LBS	CFD-GCN	GNS
Action 01	1.876	1.904	1.904	28.886	5.009
Action 02	1.838	1.867	1.868	21.693	5.865
Action 03	2.011	2.031	2.033	29.646	5.344

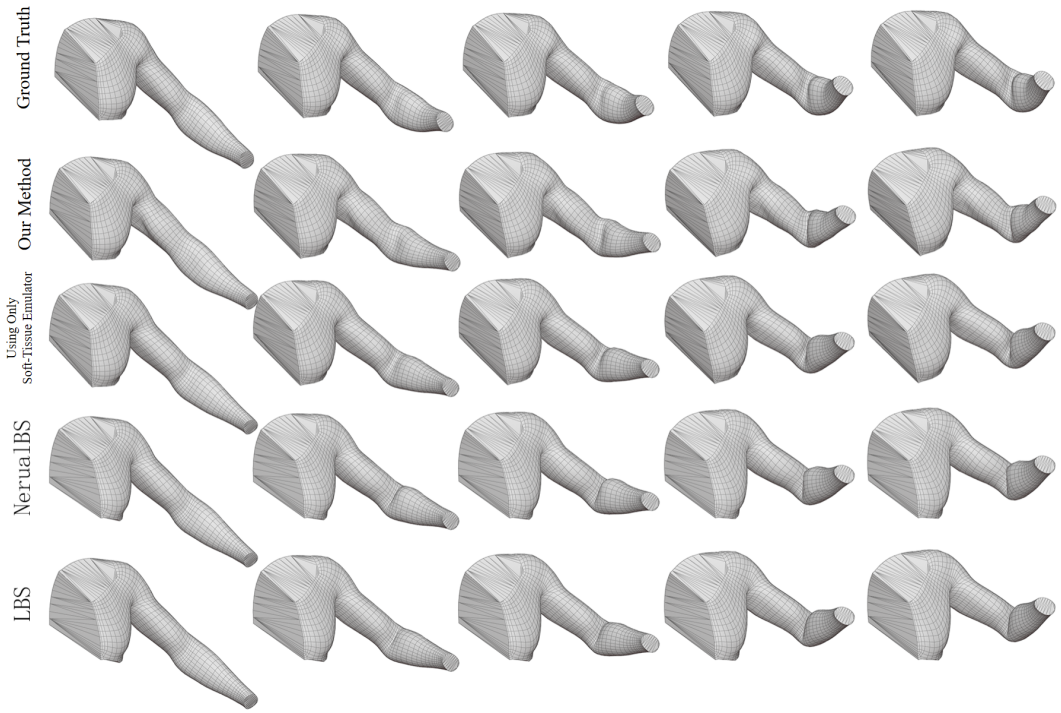

Fig. 6. Comparative experiments on the combined hierarchical method

chart indicates that, for most frames, the proposed method's results are closer to the ground truth compared to NeuralBS and LBS.

Table 3 summarizes the average roll-RMSE values for the proposed method, NeuralBS, LBS, CFD-GCN, and GNS against the ground truth. The data demonstrate that the proposed method consistently outperforms NeuralBS and LBS on this sample, producing muscle deformation predictions that are more visually faithful to the ground truth and reducing the roll-RMSE metric. NeuralBS, by focusing on refining joint deformation effects of LBS, lacks responsiveness to muscle-specific deformations. Meanwhile, general graph network frameworks such as CFD-GCN and GNS, not designed for muscle animation, fail to emulate muscle behavior effectively, leading to high roll-RMSE values and deviations from the ground truth.

The volumetric mesh of the upper limb model from Appendix A.1 was used as the prediction mesh for the neural musculoskeletal emulator, while the outer surface meshes of the upper limb and torso, derived from anatomical structures, served as the final rendering meshes. After converting

the outer surface mesh into volumetric form, it acted as the prediction mesh for the neural soft-tissue emulator. The skeletal segment validation motion set (motions 01, 02, 03) described earlier was employed, with Ziva toolkit-simulated outer surface mesh animations as ground truth. The hierarchical animation scheme proposed in this section was compared against results from the neural soft-tissue emulator alone, NeuralBS [Li et al. 2021], and LBS, using rendered images to calculate Roll-RMSE [Zheng et al. 2021] for visual fidelity assessment.

Figure 6 showcases frame captures of rendered sequences generated by different methods for motion sequence 01. The second row illustrates the hierarchical method, which produces realistic muscle deformation animations for arm flexion, including accurate biceps bulging consistent with ground truth. On this sample, the proposed method outperforms all comparisons.

Table 4. Roll-RMSE performance of the combined hierarchical method in comparative experiments

Action|RMSE ↓	Our Method	Using Only Soft-Tissue Emulator	NeuralBS	LBS
Action 01	2.712	2.706	2.850	2.844
Action 02	2.652	2.667	2.802	2.796
Action 03	2.616	2.628	2.674	2.656

Table 4 details roll-RMSE performance across motion sequences 01, 02, and 03. The combined method achieves lower roll-RMSE values, demonstrating superior visual fidelity relative to other methods. The neural soft-tissue emulator alone achieves results close to the combined method but lacks muscle contraction and expansion effects, reducing its fidelity. NeuralBS and LBS fail to capture muscle signals generated by skeletal motion, resulting in weaker performance.

We evaluates the inference performance of both emulators. The muscle emulator demonstrates superior computational throughput compared to the Ziva muscle simulation toolkit [Ziva Dynamics 2024] across motion segments 01, 02, and 03, as quantitatively validated in Table 5. Furthermore, Table 6 benchmarks the soft-tissue emulator, confirming its real-time operational capability. Notably, our inference pipeline currently operates without deliberate computational optimization – measured latency reflects baseline performance. These empirical results substantiate the framework's potential for deployment in real-time musculoskeletal animation systems, particularly in applications requiring interactive physics fidelity.

Table 5. Efficiency analysis of neural muscle emulator in comparative experiments

Method| Frame Time (s/frame)↓	Action 01	Action 02	Action 03
Our Method	0.0311	0.0290	0.0295
Ziva	0.0405	0.0318	0.0397

Table 6. Efficiency analysis of neural soft-tissue emulator with respect to sequence length (rows 1 and 2) and number of mesh vertices (rows 3 and 4).

Sequence length (frame)	153	1083	1548	7748	15,808
Duration (s/frame)	0.0213	0.0200	0.0194	0.0222	0.0250
Vertex count	1373	9126	21,595	79,743	-
Duration (s/frame)	0.023	0.081	0.180	0.682	-

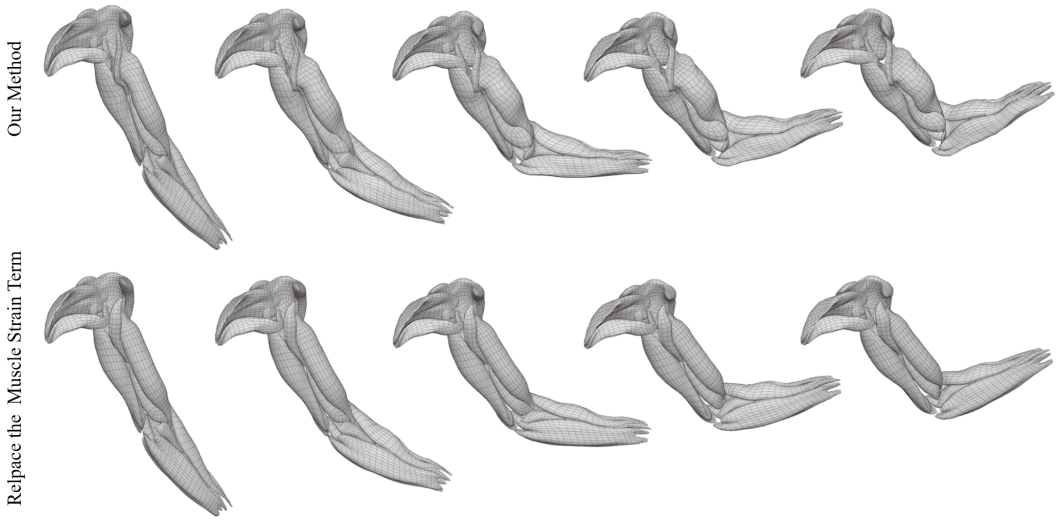

Fig. 7. Ablation experiments on the neural musculoskeletal emulator

These findings validate the hierarchical animation method's ability to outperform existing data-driven neural animation methods in visual fidelity. Unlike traditional data-driven neural methods, it avoids the need for labor-intensive ground-truth data preparation, facilitating deployment in production environments and enhancing adaptability to data outside the training range. Combined with its computational efficiency advantages over simulation toolkits, as demonstrated earlier, the proposed method is highly promising for real-time soft tissue simulation and animation systems.

4.2 Ablation Study

Using the validation upper limb model, this study evaluates the effectiveness of the biomechanical supervision module on unseen motion sequences (motions 04, 05, 06, 07, and 08, each 120 frames long). An ablation study compares the neural musculoskeletal emulator trained with the biomechanical material model supervision framework against an emulator trained with the muscle strain potential energy loss term replaced by an isotropic strain potential energy loss term.

Frame captures in Figure 7 highlight differences between the full method and the ablation method for motion 01. The full method produces realistic responses to arm flexion, with clear biceps brachii contraction and expansion. In contrast, the ablation method fails to simulate these effects, with no visible muscle contraction or expansion. These results demonstrate that the proposed biomechanical material supervision framework enables the emulator to learn accurate responses to muscle activation signals.

Figure 8 shows frame-by-frame roll-RMSE values for motions 04, 05, and 06, comparing predictions from the original method and the ablation method. The line chart indicates that the original method achieves predictions closer to the ground truth across most time steps, further validating the effectiveness of the biomechanical supervision module.

5 Conclusion

The proposed self-supervised graph neural network-based hierarchical animation generation method effectively addresses the challenges of efficient and high-fidelity soft body animation. By employing a physics-inspired self-supervised training framework, the method removes the

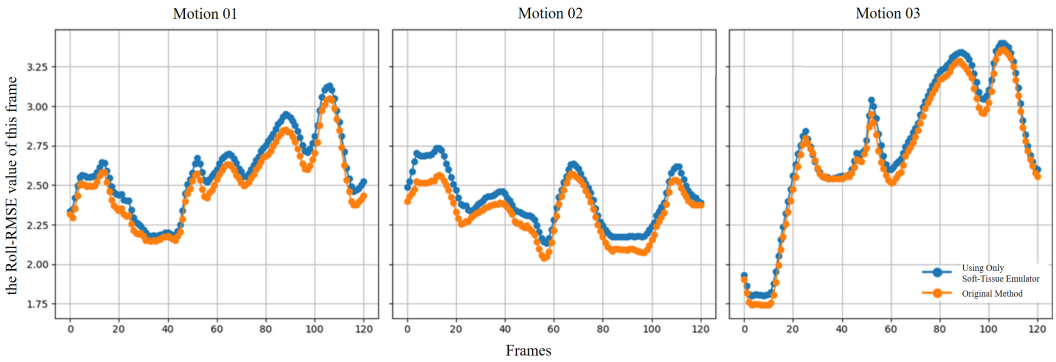

Fig. 8. Roll-RMSE metric curve for ablation experiments on the neural musculoskeletal emulator

dependency on high-quality ground-truth data, providing a robust solution for soft body skinning animation. With the musculoskeletal neural emulator and soft-tissue neural emulator capturing dynamic muscle and soft tissue characteristics, the training framework, powered by an incremental potential energy function, ensures both high efficiency and strong visual and physical fidelity. The hierarchical animation process first employs the musculoskeletal emulator to simulate muscle dynamics, followed by the soft-tissue emulator to model skin and fat responses, achieving high-fidelity animation outcomes.

During the development of this work, we identified several promising directions for future optimization and extension:

- **End-to-End Training Framework with Anatomical Embeddings.** While the current model demonstrates generalization capabilities across non-bipedal characters through customized musculoskeletal templates, future research could explore constructing an end-to-end training framework integrated with muscle anatomy embeddings. Such architecture would enable direct biomechanical feature extraction from anatomical structures, potentially eliminating manual template engineering while enhancing cross-species adaptability.
- **High-Fidelity Supervision Integration.** While the self-supervised learning framework reduces dependency on ground-truth data, our quantitative analysis reveals that the muscle emulator exhibits residual artifacts in biomechanical fidelity (as illustrated in Figure 9), manifesting as suboptimal tissue deformation responses to extreme kinematic configurations despite achieving motion-driven morphological adaptations post-convergence. Incorporating high-precision motion capture data and biomedical imaging constraints could improve simulation accuracy in extreme biomechanical scenarios. Furthermore, while no numerical instabilities were observed in current experiments, we propose implementing adaptive weighting strategies to mitigate potential stiffness-induced divergence in high-velocity movements.
- **Physics-Compliant Collision Handling.** The current framework focuses on emulator coordination rather than inter-object interactions. To address inevitable soft tissue collisions in real-world scenarios, future iterations could integrate incremental potential collision models[Li et al. 2020] – formulating contact forces as differentiable energy terms compatible with our self-supervised energy minimization paradigm. This enhancement would significantly improve physical plausibility in multi-body animation scenarios.

Through further research addressing the aforementioned challenges, the hierarchical animation method proposed in this paper is expected to significantly enhance the physical realism, applicability

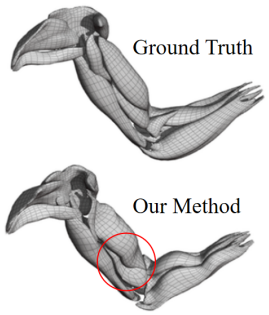

Fig. 9. Residual Artifacts

scope, and generation efficiency of human character animation, thereby meeting more extensive application requirements.

Acknowledgments

This work was partly supported by the Natural Science Foundation of China under Grant No. 62072328.

References

Dragomir Anguelov, Praveen Srinivasan, Daphne Koller, Sebastian Thrun, Jim Rodgers, and James Davis. 2005. SCAPE: shape completion and animation of people. *ACM Transactions on Graphics (TOG)* 24, 3 (2005), 408–416.

Amaury Aubel and Daniel Thalmann. 2001. Interactive modeling of the human musculature. In *Proceedings of Computer Animation*. 167–255.

Stephen W Bailey, Dave Otte, Paul Dilorenzo, and James F O'Brien. 2018. Fast and deep deformation approximations. *ACM Transactions on Graphics (TOG)* 37, 4 (2018), 1–12.

Filipe De Avila Belbute-Peres, Thomas Economon, and Zico Kolter. 2020. Combining differentiable PDE solvers and graph neural networks for fluid flow prediction. In *Proceedings of international conference on machine learning 2020*. PMLR, 2402–2411.

Blender Foundation. 2024. *Blender 3.6 LTS: A Comprehensive Toolset for 3D Animation Creation*. Blender Foundation, Amsterdam, Netherlands. https://www.blender.org/ [CP/OL].

Yadi Cao, Menglei Chai, Minchen Li, and Chenfanfu Jiang. 2023. Efficient learning of mesh-based physical simulation with bi-stride multi-scale graph neural network. In *Proceedings of the 40th International Conference on Machine Learning*. JMLR.org.

David T Chen and David Zeltzer. 1992. Pump it up: Computer animation of a biomechanically based model of muscle using the finite element method. In *Proceedings of the 19th annual conference on computer graphics and interactive techniques*. 89–98.

Yinpeng Chen, Zicheng Liu, and Zhengyou Zhang. 2013. Tensor-based human body modeling. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition 2013*. 105–112.

Nuttapong Chentanez, Miles Macklin, Matthias Müller, Stefan Jeschke, and Tae-Yong Kim. 2020a. Cloth and skin deformation with a triangle mesh based convolutional neural network. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation 2020*. 637–663.

Nuttapong Chentanez, Miles Macklin, Matthias Müller, Stefan Jeschke, and Tae-Yong Kim. 2020b. Cloth and Skin Deformation with a Triangle Mesh Based Convolutional Neural Network. *Computer Graphics Forum* 39, 8 (2020), 123–134. doi:10.1111/cgf.14107 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.14107

David Dalton, Dirk Husmeier, and Hao Gao. 2023. Physics-informed graph neural network emulation of soft-tissue mechanics. *Computer Methods in Applied Mechanics and Engineering* 417, 3 (2023), 1–13.

Tien Tuan Dao and Marie-Christine Ho Ba Tho. 2018. A Systematic Review of Continuum Modeling of Skeletal Muscles: Current Trends, Limitations, and Recommendations. *Applied Bionics and Biomechanics* 30, 1 (2018), 763–775. doi:10.1155/2018/7631818 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1155/2018/7631818

Facebook AI Research. 2024. *PyTorch: A Deep Learning Framework*. Facebook, California, USA. https://pytorch.org/ [CP/OL].

Yuan-cheng Fung. 2013. *Biomechanics: mechanical properties of living tissues*. Springer Science & Business Media.

J-P Gourret, Nadia Magnenat Thalmann, and Daniel Thalmann. 1989. Simulation of object and human skin formations in a grasping task. In *Proceedings of the 16th Annual Conference on Computer Graphics and interactive techniques*. 21–30.

Yushan Han, Yizhou Chen, Carmichael Ong, Jingyu Chen, Jennifer Hicks, and Joseph Teran. 2024. A neural network model for efficient musculoskeletal-driven skin deformation. *ACM Transactions on Graphics (TOG)* 43, 4 (2024), 12–26.

Herbert Hatze. 1978. A general myocybernetic control model of skeletal muscle. *Biological Cybernetics* 28, 3 (1978), 143–157.

Daniel Holden, Bang Chi Duong, Sayantan Datta, and Derek Nowrouzezahrai. 2019. Subspace neural physics: Fast data-driven interactive simulation. In *Proceedings of the 18th annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. 1–12.

Yongxu Jin, Yushan Han, Zhenglin Geng, Joseph Teran, and Ronald Fedkiw. 2022a. Analytically Integratable Zero-restlength Springs for Capturing Dynamic Modes unrepresented by Quasistatic Neural Networks. In *Proceedings of ACM SIGGRAPH*.

Yongxu Jin, Yushan Han, Zhenglin Geng, Joseph Teran, and Ronald Fedkiw. 2022b. Analytically Integratable Zero-restlength Springs for Capturing Dynamic Modes unrepresented by Quasistatic Neural Networks. In *ACM SIGGRAPH 2022 Conference Proceedings* (Vancouver, BC, Canada) *(SIGGRAPH '22)*. Association for Computing Machinery, New York, NY, USA, Article 37, 9 pages. doi:10.1145/3528233.3530705

Gene S Lee, Andy Lin, Matt Schiller, Scott Peters, Mark McLaughlin, Frank Hanner, and Walt Disney Animation Studios. 2013. Enhanced dual quaternion skinning for production use.. In *Proceedings of ACM SIGGRAPH 2013 Talks*. 1–3.

John P Lewis, Matt Cordner, and Nickson Fong. 2023. Pose space deformation: a unified approach to shape interpolation and skeleton-driven deformation. In *Proceedings of Seminal Graphics Papers: Pushing the Boundaries, Volume 2*. 811–818.

Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M. Kaufman. 2020. Incremental potential contact: intersection-and inversion-free, large-deformation dynamics. *ACM Trans. Graph.* 39, 4, Article 49 (Aug. 2020), 20 pages. doi:10.1145/3386569.3392425

Peizhuo Li, Kfir Aberman, Rana Hanocka, Libin Liu, Olga Sorkine-Hornung, and Baoquan Chen. 2021. Learning skeletal articulations with neural blend shapes. *ACM Transactions on Graphics (TOG)* 40, 4 (2021), 1–15.

Yuwei Li, Longwen Zhang, Zesong Qiu, Yingwenqi Jiang, Nianyi Li, Yuexin Ma, Yuyao Zhang, Lan Xu, and Jingyi Yu. 2022. Nimble: a non-rigid hand model with bones and muscles. *ACM Transactions on Graphics (TOG)* 41, 4 (2022), 1–16.

Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. 2015. SMPL: a skinned multi-person linear model. *ACM Transactions on Graphics (TOG)* 25, 1 (2015), 224–232.

Thalmann Magnenat, Richard Laperrière, and Daniel Thalmann. 1988. Joint-dependent local deformations for hand animation and object grasping. In *Proceedings of Graphics Interface'88*. 26–33.

Naureen Mahmood, Nima Ghorbani, Nikolaus F. Troje, Gerard Pons-Moll, and Michael J. Black. 2019. AMASS: Archive of Motion Capture as Surface Shapes. In *Proceedings of International Conference on Computer Vision*. IEEE, 5442–5451. doi:10.1109/ICCV.2019.00554

Joe Mancewicz, Matt Derksen, Hans Rijpkema, and Cyrus A. Wilson. 2014. Delta Mush: smoothing deformations while preserving detail. In *Digital Production Symposium*. https://api.semanticscholar.org/CorpusID:16743136

B. Markert, W. Ehlers, and N. Karajan. 2005. A general polyconvex strain-energy function for fiber-reinforced materials. *PAMM* 5, 1 (2005), 245–246. doi:10.1002/pamm.200510108

Bruce Merry, Patrick Marais, and James Gain. 2006. Animation space: A truly linear framework for character animation. *ACM Transactions on Graphics (TOG)* 25, 4 (2006), 1400–1423.

V. Modi, L. Fulton, A. Jacobson, S. Sueda, and D.I.W. Levin. 2020. EMU: Efficient Muscle Simulation in Deformation Space. *Computer Graphics Forum (CGF)* 40, 1 (2020), 234–248.

Sang Il Park and Jessica K. Hodgins. 2008. Data-driven modeling of skin and muscle deformation. *ACM Transactions on Graphics (TOG)* 27, 3 (2008), 1–6.

Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter Battaglia. 2020. Learning Mesh-Based Simulation with Graph Networks. In *International Conference on Learning Representations*. 1–13.

Gerard Pons-Moll, Javier Romero, Naureen Mahmood, and Michael J Black. 2015. Dyna: A model of dynamic human shape in motion. *ACM Transactions on Graphics (TOG)* 34, 4 (2015), 1–14.

Singiresu S Rao. 2010. *The finite element method in engineering*. Elsevier.

O Röhrle, M Sprenger, and S Schmitt. 2017. A two-muscle, continuum-mechanical forward simulation of the upper limb. *Biomechanics and Modeling in Mechanobiology* 16, 1 (2017), 743–762.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. 2020. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning*. JMLR.org, 1–14.

Igor Santesteban, Elena Garces, Miguel A. Otaduy, and Dan Casas. 2020. SoftSMPL: Data-driven Modeling of Nonlinear Soft-tissue Dynamics for Parametric Humans. *Computer Graphics Forum (CGF)* 39, 2 (2020), 65–75.

Igor Santesteban, Miguel A. Otaduy, and Dan Casas. 2022. SNUG: Self-Supervised Neural Dynamic Garments. In *Proceedings of Conference on Computer Vision and Pattern Recognition*. IEEE.

Ferdi Scheepers, Richard E Parent, Wayne E Carlson, and Stephen F May. 1997. Anatomy-based modeling of the human musculature. In *Proceedings of Computer Graphics and Interactive Techniques 1997*. 163–172.

Tobias Siebert, Olaf Till, Norman Stutzig, Michael Günther, and Reinhard Blickhan. 2014. Muscle force depends on the amount of transversal muscle loading. *Journal of Biomechanics* 47, 8 (2014), 1822–1828.

Breannan Smith, Fernando De Goes, and Theodore Kim. 2018. Stable neo-hookean flesh simulation. *ACM Transactions on Graphics (TOG)* 37, 2 (2018), 1–15.

Victor Spitzer, Michael J Ackerman, Ann L Scherzinger, and David Whitlock. 1996. The visible human male: a technical report. *Journal of the American Medical Informatics Association* 3, 2 (1996), 118–130.

Joseph Teran, Eftychios Sifakis, Geoffrey Irving, and Ronald Fedkiw. 2005. Robust quasistatic finite elements and flesh simulation. In *Proceedings of the 2005 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. 181–190.

Tianyi Wang and Shiguang Liu. 2024. Multi-scale edge aggregation mesh-graph-network for character secondary motion. *Comput. Animat. Virtual Worlds* 35, 3 (2024). doi:10.1002/CAV.2241

Xiaohuan Corina Wang and Cary Phillips. 2002. Multi-weight enveloping: least-squares approximation techniques for skin animation. In *Proceedings of the 2002 ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. 129–138.

Jeffrey A Weiss and John C Gardiner. 2001. Computational modeling of ligament mechanics. *Critical Reviews in Biomedical Engineering* 29, 3 (2001), 71–85.

Jane Wilhelms and Allen Van Gelder. 1997. Anatomically based modeling. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*. 173–180.

Ali Yilmaz. 2009. Artistic Anatomy. In *Proceedings from XIX Congress of Anatomy May*. 31–39.

Mianlun Zheng, Yi Zhou, Duygu Ceylan, and Jernej Barbic. 2021. A deep emulator for secondary motion of 3d characters. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition 2021*. 5932–5940.

Yong-Ping Zheng, Arthur FT Mak, Bokong Lue, et al. 1999. Objective assessment of limb tissue elasticity: development of a manual indentation procedure. *Journal of rehabilitation research and development* 36, 2 (1999), 71–85.

Ziva Dynamics. 2024. *Ziva VFX: Character Muscle and Skin Simulation Software.* Ziva Dynamics, Vancouver, Canada. https://zivadynamics.com/ [CP/OL].

A Appendix

A.1 Skeletal-muscle Parameterization Model

To facilitate model training, we propose a bone-muscle model based on bone length parameterization, incorporating the following components: bone segments $\mathbf{B} = \{\mathbf{s}_i\}$ and their surface meshes $\{\mathcal{B}_i\}$, as well as muscle bundle pathlines $\mathbf{M} = \{\mathbf{m}_j\}$ and their surface meshes $\{\mathcal{M}_j\}$. Each bone segment $\mathbf{s}_i$ is defined by two joint points J_i and a segment length L_i, while each muscle pathline $\mathbf{m}_j$ consists of path points P_j. In the reference state, joint points, path points, and bone lengths are denoted as $\{J_i^0\}$, $\{P_j^0\}$, and $\{L_i^0\}$, respectively.

Parameterization of Bone Segments. The position of the i-th joint J_i is derived from its parent joint J_{i-1} and the bone length L_{i-1} in the reference state:

$$J_i = J_{i-1} + L_{i-1} \cdot \mathbf{d}_{i-1}^0,$$

where $\mathbf{d}_{i-1}^0$ is the unit direction vector of the $(i-1)$-th bone segment in the reference state.

Parameterization of Muscle Path Points. If a path point P_j is influenced by multiple bone segments, its position is computed as:

$$P_j = J_i + \sum_k w_{jk} \cdot \frac{L_k}{L_k^0} \cdot \mathbf{v}_{jk}^0,$$

where $\mathbf{v}_{jk}^0 = P_j^0 - J_k^0$, and w_{jk} represents the influence weight of the k-th bone segment on the path point. The weights satisfy $\sum_k w_{jk} = 1$ and can be determined anatomically or through inverse-distance weighting based on the distances between path points and joint points.

Parameterization of Bone and Muscle Surface Meshes. The transformation matrix of the i-th bone segment in the reference state is:

$$\mathbf{T}_i = \tilde{\mathbf{R}}_i \cdot \mathbf{S}_i \cdot \tilde{\mathbf{T}}_{\text{trans},i},$$

where $\tilde{\mathbf{R}}_i$ and $\tilde{\mathbf{T}}_{\text{trans},i}$ are the reference rotation and translation matrices, respectively. When the bone length scales by $\alpha_i = \frac{L_i}{L_i^0}$, $\mathbf{S}_i$ is a diagonal matrix constructed using α_i.

For any vertex in the skeletal surface mesh $\mathcal{B}_i$, its position in the current frame can be expressed as:

$$\mathbf{v'}_i^B = \sum_j w_j^B \cdot \mathbf{T}_j \cdot \tilde{\mathbf{v}}_i^B,$$

where w_j^B represents the weight of influence from the j-th skeletal segment, $\tilde{\mathbf{v}}_i^B$ is the vertex's coordinate in the reference state, and $\mathbf{T}_j$ is the transformation matrix of the j-th skeletal segment in the current animation state. When the skeletal length or pose changes, the vertex position is updated accordingly.

The deformation of vertices in the muscle surface mesh $\mathcal{M}_j$ follows the same principle.

A.2 biological parameter precomputation module

The biological parameter precomputation module (Algorithm 1) computes the activation values of muscle bundles, fiber direction vectors, and tendon-muscle attribute values as part of the physical parameter vector.

Algorithm 1 Biological Parameter Precomputation

Require: Reference points $\{P_i^0\}$, animation points $\{P_i^t\}$, muscle mesh vertices $\{v_i\}$, Gaussian parameter σ.

Ensure: Material attributes M^v, activation state α, fiber directions $\mathbf{a}_0(v_i)$.

Initialize $M^v \leftarrow 0$, $\alpha \leftarrow 0$, $\mathbf{a}_0 \leftarrow \{\}$.
for each vertex v_i in $\{v_i\}$ **do**
  $d^v_{\min} \leftarrow \min_{P_i^0} \|v_i - P_i^0\|$
  $M^v \leftarrow \exp\left(-(d^v_{\min})^2/(2\sigma^2)\right)$
end for
$L_0 \leftarrow \sum_{j=1}^{N_p-1} \|P_j^0 - P_{j+1}^0\|$
$L_t \leftarrow \sum_{j=1}^{N_p-1} \|P_j^t - P_{j+1}^t\|$
if $L_t \le 0.75\, L_0$ **then**
  $\alpha \leftarrow 1$
else if $0.75\, L_0 < L_t < L_0$ **then**
  $\alpha \leftarrow 1 - \frac{L_t - 0.75\, L_0}{0.25\, L_0}$
else
  $\alpha \leftarrow 0$
end if
Use cubic spline $C(t)$, $t \in [0,1]$ to fit path.
$\mathbf{t}(t) \leftarrow \frac{dC(t)}{dt}$ (path tangent).
for each vertex v_i in $\{v_i\}$ **do**
  $d(t) \leftarrow \|v_i - C(t)\|^2$
  $t_{\min} \leftarrow \arg\min_{t\in[0,1]} d(t)$
  $\mathbf{a}_0(v_i) \leftarrow \mathbf{t}(t_{\min})$
end for
return $\{\mathbf{a}_0(v_i)\}$

A.3 Assembling Stiffness Matrix

K is the stiffness matrix, assembled from the element stiffness matrices K_e:

$$K_e = V_e B_e^\top D_e B_e, \tag{11}$$

Here, V_e represents the volume of the element, with the coordinates of the i-th vertex denoted as X_i, Y_i, Z_i. The element volume is expressed by the mesh volume as:

$$V_e = \frac{1}{6}\left|\det\begin{bmatrix} X_2 - X_1 & X_3 - X_1 & X_4 - X_1 \\ Y_2 - Y_1 & Y_3 - Y_1 & Y_4 - Y_1 \\ Z_2 - Z_1 & Z_3 - Z_1 & Z_4 - Z_1 \end{bmatrix}\right|, \tag{12}$$

B_e is the element strain-displacement matrix, assembled in the reference configuration. It is typically derived as follows:

$$B_e = \begin{bmatrix} b_1 & 0 & 0 & b_2 & 0 & 0 & b_3 & 0 & 0 & b_4 & 0 & 0 \\ 0 & c_1 & 0 & 0 & c_2 & 0 & 0 & c_3 & 0 & 0 & c_4 & 0 \\ 0 & 0 & d_1 & 0 & 0 & d_2 & 0 & 0 & d_3 & 0 & 0 & d_4 \\ c_1 & b_1 & 0 & c_2 & b_2 & 0 & c_3 & b_3 & 0 & c_4 & b_4 & 0 \\ 0 & d_1 & c_1 & 0 & d_2 & c_2 & 0 & d_3 & c_3 & 0 & d_4 & c_4 \\ d_1 & 0 & b_1 & d_2 & 0 & b_2 & d_3 & 0 & b_3 & d_4 & 0 & b_4 \end{bmatrix}, \tag{13}$$

Here, b_i, c_i, d_i are the derivatives of the shape functions with respect to the spatial coordinates x, y, z:

$$b_i = \frac{1}{6V_e}\left((Y_j - Y_k)(Z_l - Z_k) - (Y_l - Y_k)(Z_j - Z_k)\right), \tag{14}$$

$$c_i = \frac{1}{6V_e}\left((Z_j - Z_k)(X_l - X_k) - (Z_l - Z_k)(X_j - X_k)\right), \tag{15}$$

$$d_i = \frac{1}{6V_e}\left((X_j - X_k)(Y_l - Y_k) - (X_l - X_k)(Y_j - Y_k)\right), \tag{16}$$

D_e is the elasticity matrix of the material, simplified for isotropic materials as:

$$D_e = \frac{E}{(1+\nu)(1-2\nu)}\begin{bmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix}. \tag{17}$$

where E is Young's modulus and ν is Poisson's ratio.